EEST N°1 De Vicente López

Laboratorio de Programación (LPR) — 5° 3°
A-B

Informe de Proyecto

Control de Acceso por Edad — Actividad 4: Manipulación de Cadenas y Punteros en C/C++

Alumno: Lucas Del Pino

Curso: 5° 3° A-B

Docente: Prof. York

Fecha de entrega: 15 de octubre de 2026

Repositorio GitHub: https://github.com/Pin0l33/cadenaspunteros.git

1. Introducción

El presente informe técnico expone el desarrollo de la Actividad 4 correspondiente a la asignatura Laboratorio de Programación (LPR), dictada durante el primer cuatrimestre de 2026. La investigación y práctica implementada se fundamentan en el diseño e implementación de un sistema de control de acceso por edad en lenguaje C++. El propósito primordial consistió en gestionar datos directamente sobre la memoria RAM a través de punteros y operaciones de desreferenciación. Un puntero se define formalmente como una variable cuyo propósito específico es almacenar la dirección física de memoria asociada a otra variable. La desreferenciación (ejecutada mediante el operador asterisk *) permite leer, manipular y actualizar de manera directa el contenido alojado en dicha ubicación de memoria.

2. Estructura del repositorio

El proyecto ha sido estructurado en la plataforma GitHub respetando meticulosamente las directrices académicas establecidas por la cátedra:

`/CADENASPUNTEROS/ - .gitignore, LICENSE, README.md, docs/, src/, capturas/`

El directorio `docs/` alberga la documentación formal y el presente informe en formato PDF, la carpeta `src/` contiene el código fuente principal (`main.cpp`), y el directorio `capturas/` reúne las evidencias gráficas de la ejecución, junto con el esquema representativo de la memoria.

3. Captura del código fuente (Visual Studio Code)

A continuación, se exhibe el código fuente correspondiente al archivo `main.cpp`, en el cual se valida la autenticidad de la autoría del estudiante mediante la salida del programa. En el diseño algorítmico, el puntero `p` almacena la dirección de memoria de la variable `edad`; consecuentemente, cada operación de lectura, evaluación y comparación lógica se efectúa estrictamente mediante la desreferenciación `*p`.

```cpp
/*
    ================================================================
    Materia: Laboratorio de Programación (LPR)
    E.E.S.T. Nº 1 "Eduardo Ader" – Vicente López
    Curso: 5º Año 3º División A
    Profesor: Prof. York, Prof. Ganduglia
    Archivo: main.cpp
    Objetivo: Control de accesos operando directamente sobre la RAM con punteros.
    ================================================================
*/

#include <iostream>

using namespace std;

int main() {
    int edad = 0;
    int* p = &edad; // El puntero 'p' guarda la dirección de memoria física de 'edad'

    // REGLA OBLIGATORIA (EVITA EL PLAGIO): nombre y apellido reales.
    cout << "=====================================================" << endl;
    cout << "   CONTROL DE ACCESO RAM - ESTUDIANTE: Lucas Del Pino " << endl;
    cout << "=====================================================" << endl;

    cout << "=> Ingrese su edad: ";

    // 1. CAPTURA MEDIANTE DESREFERENCIACIÓN
    //    'cin >> *p;' guarda el número directamente en el casillero de 'edad'.
    //    'cin >> p;' (sin asterisco) intentaría sobrescribir la dirección del puntero.
    cin >> *p; // Captura el dato guardándolo directo en la memoria apuntada

    cout << "\n--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---" << endl;

    // 2. CONDICIONAL CON PUNTERO DESREFERENCIADO
    //    '*p >= 18' evalúa el valor alojado en la memoria.
    //    Mostrar solo 'p' imprime la dirección hexadecimal (ej: 0x61fe1c).
    if (*p >= 18) {
        cout << "[ACCESO APROBADO] El usuario es mayor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    } else {
        cout << "[ACCESO RECHAZADO] Menor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    }

    cout << "=====================================================" << endl;
    return 0; // Finalización exitosa
```

4. Captura de la ejecución (Terminal)

4.1 Compilación y ejecución del programa en C++

El programa fue compilado utilizando la herramienta GCC mediante el comando `g++ src/main.cpp -o src/cadenaspunteros.exe` y ejecutado en entorno CLI mediante `.\src\cadenaspunteros.exe`. Se realizaron pruebas de validación con valores mayores a 18 años (resultado de acceso aprobado, Figura 2) y menores a 18 años (resultado de acceso denegado, Figura 3). Cabe destacar que la dirección hexadecimal asignada varía dinámicamente en cada ciclo de ejecución del sistema operativo.

Figura 2

Ejecución con edad 21: acceso aprobado y dirección hexadecimal

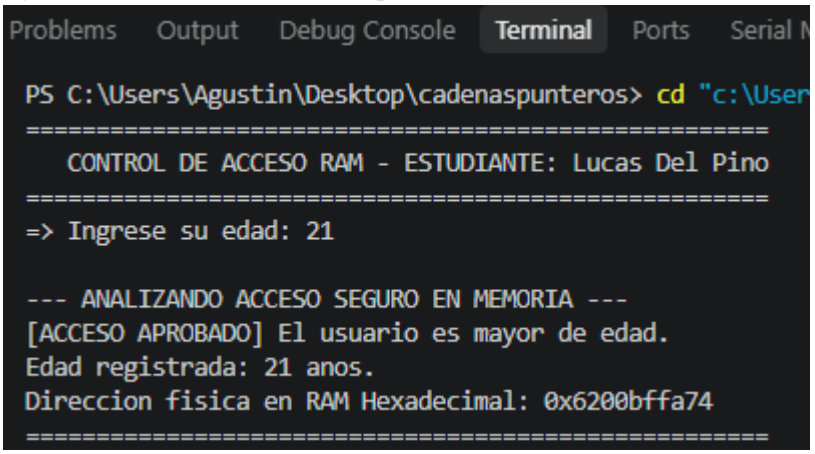
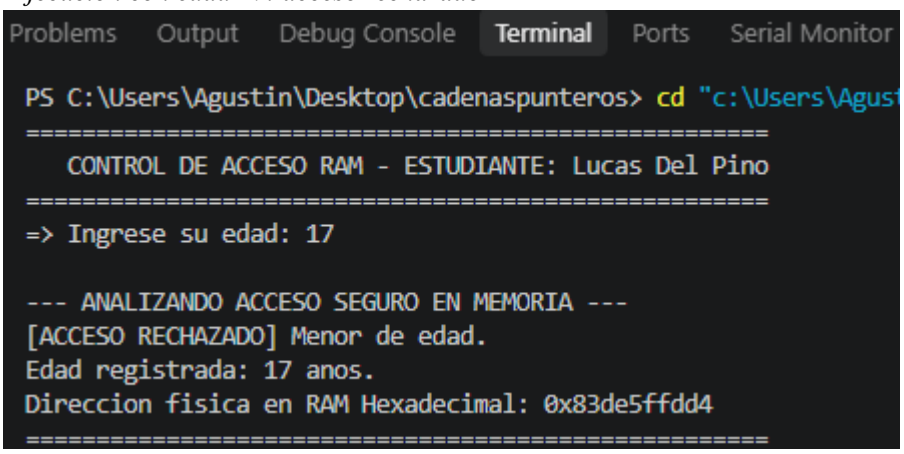


Figura 3

Ejecución con edad 17: acceso rechazado

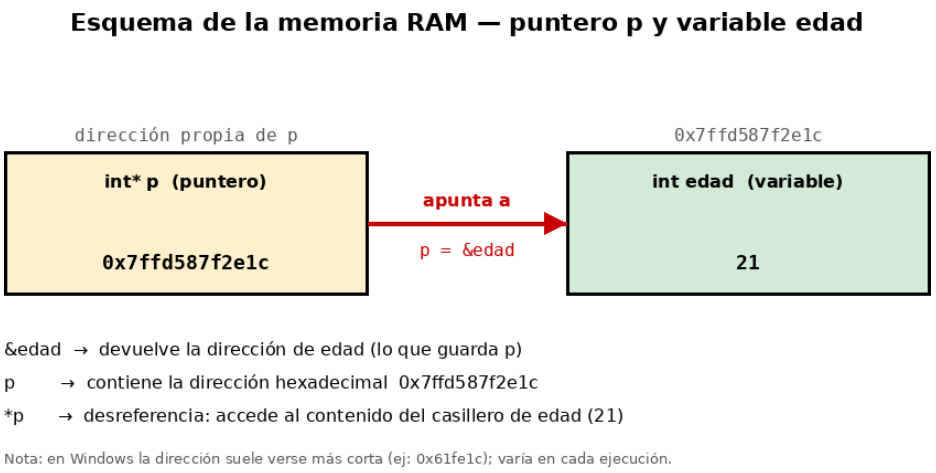


4.2 Esquema de la memoria RAM

La Figura 4 ilustra la arquitectura lógica del puntero `p`, detallando cómo este almacena la dirección hexadecimal de la celda de memoria donde reside la variable `edad`.

Figura 4

Esquema de memoria: el puntero p apunta a la variable edad



5. Cuestionario técnico de control

Pregunta 1: Diferencia entre & y *

*¿Qué valor imprime en pantalla la instrucción cout << p; comparada con cout << *p;?*

`cout << p;` imprime la dirección de memoria física en formato hexadecimal asociada a la variable `edad` (por ejemplo, `0x7ffd587f2e1c` en Linux o `0x61fe1c` en entornos Windows). En contraposición, la sentencia `cout << *p;` desreferencia la dirección del puntero, obteniendo y desplegando el valor entero almacenado en dicho bloque de memoria (21 en la prueba documentada). En síntesis, el operador `&edad` retorna la ubicación exacta en memoria RAM, mientras que `*p` accede y recupera el dato resguardado en ella.

Pregunta 2: Error crítico al guardar entrada

*¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?*

Dado que el objeto `p` es declarativamente un tipo de dato `int*`, su asignación directa representa la dirección de memoria y no un dato escalar numérico. Intentar ejecutar `cin >> p;` provoca un fallo de tipeo en tiempo de compilación, puesto que el flujo de entrada `cin` no posee una sobrecarga válida para almacenar datos directamente sobre un puntero. Si tal operación se forzase mediante casting explícito, el puntero sobreescribiría su dirección por una punción arbitraria, generando accesos indebidos a zonas no reservadas y el consecuente fallo catastrófico del programa (segmentation fault). Por ende, la utilización de `cin >> *p;` es imperativa: el operador asterisco garantiza que la escritura orientada por el usuario se realice sobre la dirección apuntada por `p`, es decir, modificando directamente la variable primitiva `edad`.

Pregunta 3: Formato hexadecimal

¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

La arquitectura de hardware y el sistema operativo representan la memoria física a través de patrones binarios. La notación hexadecimal es óptima dado que cada dígito de base 16 mapea de manera exacta e inequívoca a un grupo de 4 bits (nibble). Por consiguiente, un byte se expresa de forma concisa mediante 2 dígitos hexadecimales, y las direcciones de 32 o 64 bits se representan mediante 8 o 16

caracteres respectivamente. La conversión entre notación binaria y hexadecimal es directa y biunívoca, cualidad de la que carece el sistema decimal. El prefijo estándar `0x` denota explícitamente dicha base numérica. Esta representación simplifica la lectura humana sin alterar la naturaleza binaria subyacente empleada internamente por la unidad central de procesamiento.

6. Sección personal: variable común vs. puntero

Existe una diferencia estructural entre una variable primitiva y un puntero. Una variable convencional como `edad` almacena de forma directa un valor escalar en memoria. Por su parte, un puntero como `p` contiene la referencia de memoria de dicha variable, posibilitando la manipulación indirecta de sus datos. Operar mediante `*p` es funcionalmente equivalente a intervenir sobre la variable `edad`; cualquier alteración efectuada a través del puntero impacta inmediatamente sobre la variable original. Este principio establece los cimientos para mecanismos avanzados en C++, tales como la transmisión de parámetros por referencia, el manejo de estructuras dinámicas y la asignación dinámica de memoria, otorgando al programador un nivel de control sobre la infraestructura del sistema que suele abstraerse en lenguajes de alto nivel.

7. Conclusión

La realización del presente trabajo práctico permitió consolidar la comprensión teórica y práctica respecto a la organización de la memoria RAM y la interacción entre variables, direcciones físicas y punteros a través de los operadores de dirección (`&`) y desreferenciación (`*`). El sistema de control de acceso por edad fue implementado y verificado satisfactoriamente bajo distintos escenarios de prueba, revalidando la trascendencia de la gestión eficiente de memoria de bajo nivel en el desarrollo informático.